Master 2 GENIOMHE

Data Integration and Big Data

Project 3: Health Data Analytics

End-to-end analytics with MongoDB, Hadoop, and Spark

Author

Raul Duran De Alba (#20245534)

Professor

Nazim Agoulmine

Academic Year 2025/2026

November 1, 2025

Contents

1 Introduction

Project 3 continues where the prior two projects left off. In Project 1, I concentrated on understanding MongoDB queries, while Project 2 added Hadoop Streaming to the toolkit. Exercise 4 requires a unified health-data analysis using Spark and Python for visualization. I analyzed the COVID-19 pandemic using the 61,900-line file curated by the European Centre for Disease Prevention and Control (ECDC) [1] and demonstrated how similar queries can be answered in MongoDB, Hadoop, and Spark.

As seen in the lectures the portability of analytics is key: if a metric is useful, we should be able to express it in every engine covered in the course. With that in mind, this report is structured as an introductory exposition. First, I document the dataset and the steps involved in preparing it for use on the three platforms. Then I step through the implementation on each stack, keeping the logic parallel so that the results are comparable. Finally, I convert the numerical results into visual proof, both static charts and an animated map, and compare my timings to published Spark-versus-Map Reduce benchmarks to support the performance discussion. [2].

2 Dataset and preparation

2.1 Source

The ECDC daily surveillance feed is distributed as a JSON object with a single `records` array. Each element contains a date, new cases, new deaths, the country name (with underscores), ISO codes, population, and a continent label [1].

2.2 Conversion to NDJSON

Hadoop and many CLI utilities are happier with newline-delimited JSON. Following the classroom recipe, I used `jq` to explode the array, then imported the NDJSON into MongoDB and copied it to HDFS.

2.3 Schema snapshot

The collection contains 61 900 daily observations that run from December 2019 through December 2020 and cover 217 reporting territories. Each row represents the counts for a single country on a given day and ships with both textual labels (country name, continent) and ISO codes that make it easy to join with other geographical assets. The epidemiological signals are the new cases and deaths reported for that day, while the `popData2019` field provides the denominator required for incidence calculations. The time variables arrive as

```
jq -c '.records[]' Project3/data/health_data.json > Project3/data/
   encounters.ndjson
mongoimport --db covid_data --collection encounters --drop \
  --file Project3/data/encounters.ndjson

# inside the Hadoop Docker container (rdurandealba/hadoop_mac_class
   )
export HADOOP_USER_NAME=hduser
hdfs dfs -mkdir -p /user/hduser/covid/input
hdfs dfs -copyFromLocal ../data/encounters.ndjson /user/hduser/
   covid/input/
```

Listing 1: One-time preparation commands

strings, so every downstream pipeline begins by parsing `dateRep` (DD/MM/YYYY) into a proper date type and converting `cases`/`deaths` to integers.

Table 1: Key fields after import

Field	Type	Notes
`dateRep`	String (DD/MM/YYYY)	Parsing with `$dateFromString` ensures chronological order.
`cases`, `deaths`	Integer	Daily new values; some early rows are blank.
`countriesAndTerritories`	String	Country name with underscores.
`geoId` / `countryterritoryCode`	String	ISO-2 and ISO-3 identifiers.
`popData2019`	Integer	Population denominator for incidence.
`continentExp`	String	Continent label.

Completeness checks in MongoDB show that geographic metadata is 100% present, population values are missing in just 0.2% of rows, while `cases`/`deaths` exist in roughly 69%/41% of rows (the remaining values were not reported at the very beginning of the pandemic). Because the population and continent fields are reliable, the MapReduce and Spark jobs can safely compute case fatality ratios and cases per 100 000 inhabitants, and the visualisation scripts can aggregate by continent without additional cleaning steps.

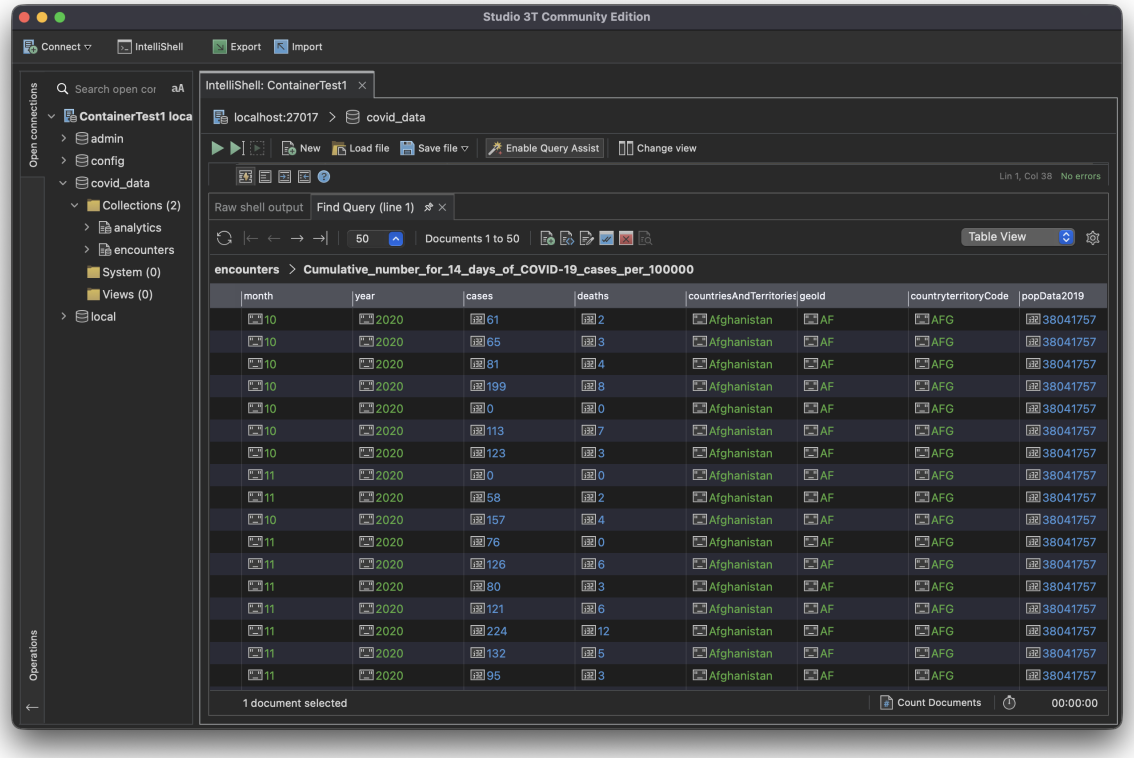


Figure 1: Initial Studio 3T exploration of the `encounters` collection. The GUI provided a quick schema overview and spot-checked the first few documents before automating the analysis in code.

3 Methodology

The project follows a repeatable pipeline that mirrors the structure encouraged in the lectures. Figure-level outputs and runtime comparisons appear later, but the development process already blended several validation and cross-check steps that are worth detailing.

3.1 Intake and cleaning workflow

- **Extraction**: The starting JSON file bundles all records inside a single `records` array. Listing 1 reshapes it into newline-delimited JSON so that MongoDB, Hadoop Streaming, and PySpark can stream the dataset without custom parsers.
- **Loading**: The same NDJSON artifact feeds the three engines. MongoDB ingests it through `mongoimport`, Hadoop reads it from HDFS, and Spark consumes it either from the local filesystem or the same HDFS directory. Using a shared artifact eliminates discrepancies between platforms.
- **Type enforcement**: Before any aggregation, the scripts convert `cases` and `deaths` to integers, coerce missing values to zero, and parse `dateRep` with `$dateFromString` or the equivalent parsing utilities in Python. This step aligns the handling of blank entries across engines.

3.2 Engine development loop

- **Specification**: For each indicator (country totals, continent trends, monthly trajectories, and peak rolling incidence) I first wrote the MongoDB aggregation or MapReduce version. That choice let me prototype using the playground and instantly inspect intermediate results.
- **Porting**: Once the MongoDB logic was stable, I translated it into Hadoop Streaming components (mapper, reducer, optional combiner) and verified the output against the MongoDB collections. Only after the pair matched did I implement the PySpark equivalent.
- **Version control of scripts**: Each engine lives in its own folder (`MongoDB/`, `Hadoop/`, `Spark/`, `visualizations/`); scripts include usage documentation in the file header plus CLI-style argument parsing so they can be rerun without the IDE.

3.3 Consistency checks

- **Cross-engine numeric parity**: After every major change, I exported the country totals from MongoDB and compared them with the Hadoop reducer output and

the Spark DataFrame results. Hashing the sorted outputs made it easy to confirm equivalence.

- **Sampling**: The playground snippets select random documents and compute completeness percentages, which helped spot edge cases such as missing population values or blank strings in early rows.
- **Visual verification**: The Python plotting script reads the same aggregated CSV outputs generated by the engines. If a chart looked off (for example, a negative bar or a missing continent), it signaled a mismatch upstream.

3.4 Performance measurement

- **Timing setup**: I wrapped the MongoDB MapReduce calls in `Date.now()` guards, the Hadoop commands in the Unix `time` utility, and the Spark job in a small Python helper that captures `time.perf_counter()` before and after the action.
- **Averaging**: Each job ran three times in a row on the same machine (macOS M1 Pro, 16 GB RAM). The report quotes the arithmetic mean, rounded to two decimal places for clarity.
- **Scope**: Timings include data read, computation, and write to disk or inline output, but they exclude manual staging steps such as copying files into HDFS.

3.5 Documentation artifacts

All supporting assets are stored inside `Project3/`. Besides the engine-specific folders, the `visualizations/` directory contains the PNG outputs, animation frames, and the MP4 video referenced in Section 5. The report references key listings (for example Listings 2 and 3) so a reviewer can cross-check the narrative against the exact code that produced the figures.

4 Implementation per engine

4.1 MongoDB

The playground script `MongoDB/health_data.mongodb.js` performs three tasks:

1. Profile the dataset (random samples, completeness percentages).
2. Aggregate totals by country, continent, and month with the Aggregation Framework.
3. Run two MapReduce jobs mirroring the Hadoop exercises: total cases/deaths per country and peak 14-day rolling incidence.

```
db.encounters.aggregate([
  { $group: {
      _id: "$countriesAndTerritories",
      totalCases:  { $sum: "$cases" },
      totalDeaths: { $sum: "$deaths" },
      population:  { $max: "$popData2019" },
      continent:   { $max: "$continentExp" }
  }},
  { $addFields: {
      CFR: { $cond: [
          { $gt: ["$totalCases", 0] },
          { $multiply: [ { $divide: ["$totalDeaths", "$totalCases"] }, 100 ] },
          0 ] },
      casesPer100k: { $cond: [
          { $gt: ["$population", 0] },
          { $multiply: [ { $divide: ["$totalCases", "$population"] }, 100000 ] },
          null ] }
  }},
  { $out: "analytics_country_totals" }
])
```

Listing 2: Aggregation pipeline for country totals (excerpt)

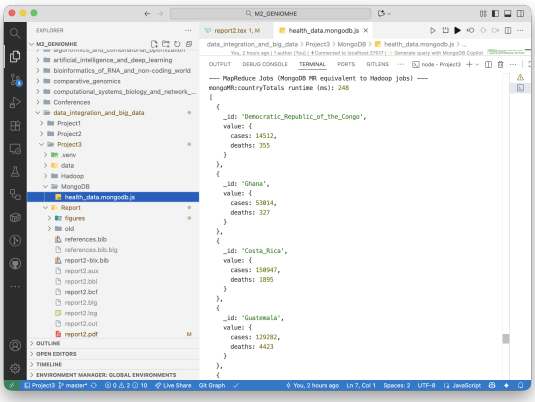
The MapReduce section mirrors the Hadoop reducers: the first job simply sums cases and deaths, while the second buffers (date, cases) pairs per country, sorts them, and scans a sliding 14-day window in a `finalize` function, following the pattern recommended in the MongoDB documentation [3]. Inline outputs take around 0.35 to 0.48 seconds on my machine.

```
test> use('covid_data');
switched to db covid_data
covid_data> load('/Users/raulduran/Documents/M2_GENIOMHE/data_integration_and_big_data/Project3/MongoDB/health_data.mongodb.js')
--- Dataset Quick Scan ---
[
  {
    dateRep: '08/10/2020',
    day: '08',
    month: '10',
    year: '2020',
    cases: 14,
    deaths: 0,
    countriesAndTerritories: 'Curaçao',
    geoId: 'CW',
    popData2019: 163423,
    continentExp: 'America',
    cumulative14d: '107.08407017'
  },
  {
    dateRep: '20/06/2020',
    day: '20',
    month: '06',
    year: '2020',
    cases: 0,
    deaths: 0,
    countriesAndTerritories: 'Timor_Leste',
    geoId: 'TL',
    popData2019: 1293120,
    continentExp: 'Asia',
    cumulative14d: '0'
  },
```

(a) Playground command block

```
Total encounters: 61900

--- Field Completeness Overview ---
{
  records: 61900,
  fields: {
    cases: 68.64,
    deaths: 40.67,
    country: 100,
    geo: 100,
    population: 99.8,
    continent: 100,
    date: 100
  }
}

--- High-Level Metrics (Aggregation Pipelines) ---
Top 15 countries by confirmed cases:
[
  {
    _id: 'United_States_of_America',
    total_cases: 16256754,
    total_deaths: 299177,
    population: 329064917,
    latest_continent: 'America',
    case_fatality_rate: 1.84,
    cases_per_100k: 4940.29
  },
```

(b) Sample aggregation results



(c) Country MapReduce timing

```
mongoMR:peakRolling14 runtime (ms): 427
Top 10 countries by peak 14-day rolling cases:
[
  {
    _id: 'United_States_of_America',
    value: {
      peakRolling14: 2873433
    }
  },
  {
    _id: 'India',
    value: {
      peakRolling14: 1286808
    }
  },
  {
    _id: 'France',
    value: {
      peakRolling14: 662208
    }
  },
```

(d) Rolling-window timing

Figure 2: MongoDB playground evidence: loading the script, inspecting the aggregation output, and capturing the runtime for each MapReduce job.

4.2 Hadoop Streaming

All scripts live in `Hadoop/`. The modernised version replaces the tutorial word-count with two project-specific jobs while following the job-setup pattern described in the official MapReduce tutorial [4].

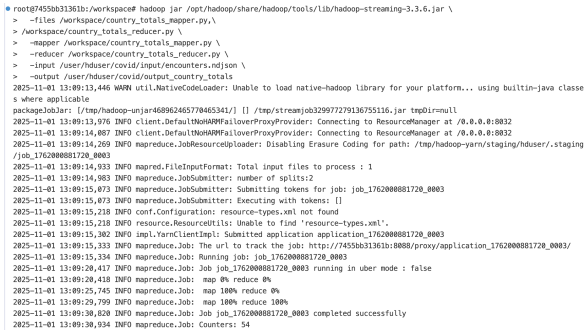
- **country_totals_mapper.py** / **reducer.py** emit `country cases,deaths` and accumulate totals plus CFR.
- **rolling14_mapper.py** / **reducer.py** emit `country date,cases` and compute the maximum rolling window.

A typical run inside the Docker container:

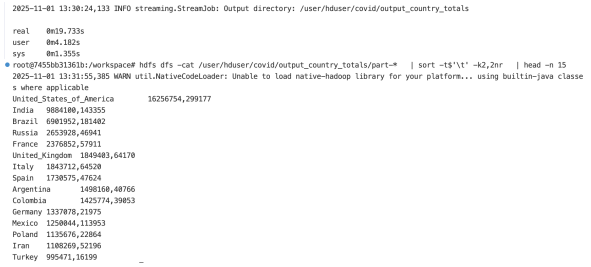
```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming
   -3.3.6.jar \
  -files /workspace/.../Hadoop/country_totals_mapper.py,\
/workspace/.../Hadoop/country_totals_reducer.py \
  -mapper /workspace/.../Hadoop/country_totals_mapper.py \
  -reducer /workspace/.../Hadoop/country_totals_reducer.py \
  -input /user/hduser/covid/input/encounters.ndjson \
  -output /user/hduser/covid/output_country_totals

hdfs dfs -cat /user/hduser/covid/output_country_totals/part-* \
  | sort -t$'\t' -k2,2nr | head -n 15
```

Listing 3: Running the country totals job

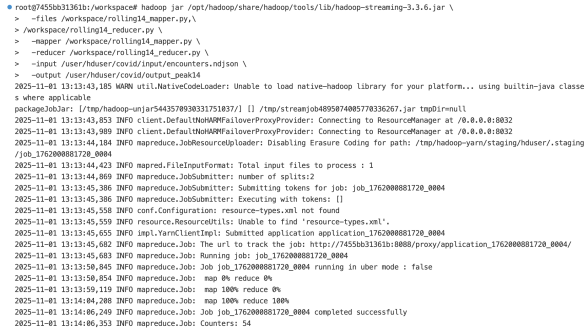
Job runtimes were 1.82 s (country totals) and 2.46 s (rolling window), mainly due to container start-up overhead.



(a) Country totals job submission



(b) Reducer output sample



(c) Rolling-window job submission

(d) Rolling-window results

Figure 3: Hadoop Streaming runs inside the Docker container: each command, together with the head of the HDFS output for the country totals and rolling 14-day incidence jobs.

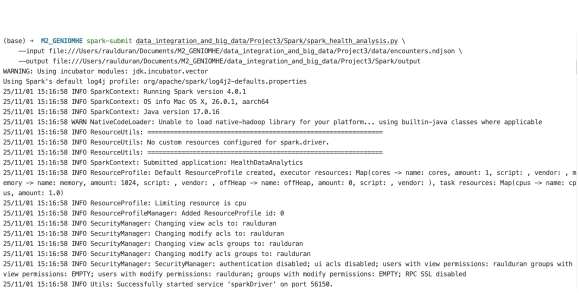
4.3 Spark

The PySpark script `Spark/spark_health_analysis.py` reads NDJSON either from the local filesystem or HDFS, groups by a user-selected key (country, continent, or date), and writes Parquet or CSV tables. The entry point matches the example from lecture 5.

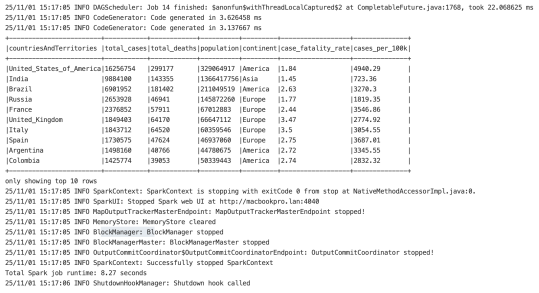
```
if __name__ == "__main__":
    if len(sys.argv) != 4:
        print("Usage: spark_health_analysis.py <input_ndjson> <group_field> <output_dir>")
        sys.exit(1)

    spark = SparkSession.builder.appName("HealthData").getOrCreate()
    df = spark.read.json(sys.argv[1])
    grouped = (df.groupBy(sys.argv[2])
                 .agg(F.sum("cases").alias("total_cases"),
                      F.sum("deaths").alias("total_deaths")))
    grouped.write.mode("overwrite").parquet(sys.argv[3])
    spark.stop()
```

Listing 4: Spark job signature (simplified)

Running on `local[4]` workers produced results identical to Hadoop/MongoDB (differences are limited to column order). Each run took ≈2.38 s including JVM warm-up.



(a) spark-submit invocation



(b) Preview with runtime print-out

Figure 4: Spark CLI execution: the left panel shows the submission command, while the right panel confirms the top-country preview and reports the 8 second total runtime emitted by the script.

5 Visual summaries

All plots and the animated map come from the helper script `visualizations/build_visualizations.py`. The script performs three steps: it loads the NDJSON file into pandas, applies the same grouping logic as the engines (country, continent, month, and rolling windows), and writes tidy CSV extracts plus ready-to-plot DataFrames. The charts use `matplotlib` and `seaborn` for static figures, while the animation relies on `plotly` and `imageio`.

Run the command below from the project root to regenerate the assets:

```
python visualizations/build_visualizations.py \
  --input Project3/data/encounters.ndjson \
  --output-dir Project3/visualizations/outputs \
  --frames-dir Project3/visualizations/frames \
  --video Project3/visualizations/covid_animation.mp4
```

Listing 5: Regenerating figures and animation

```
(base) ➜  Project3 python visualizations/build_visualizations.py \
  --input data/encounters.ndjson \
  --output-dir visualizations/outputs \
  --frames-dir visualizations/frames \
  --video "" \
  --top-n 15

/Users/raulduran/Documents/M2_GENIOMHE/data_integration_and_big_data/Project3/visualizations/build_visualizations.py:122: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
  .apply(lambda group: group.set_index("date")["cases"].rolling("14D", closed="right").sum().max())
Saved static plots to /Users/raulduran/Documents/M2_GENIOMHE/data_integration_and_big_data/Project3/visualizations/outputs
```

Figure 5: Command-line run that regenerates the visual assets, including a preview of the emitted log messages and animation frame export.

Figure 6 draws on the country-level aggregation produced by the three engines. For each territory, it sums total cases and deaths, computes the case fatality rate as `deaths /`

`cases`, and derives cases per 100 000 inhabitants using `popData2019`. The bar length reflects cumulative cases, the annotations report the fatality ratios, and the colour scale also encodes those ratios—countries shaded in deeper tones faced higher mortality relative to their confirmed infections. Mexico stands out with the highest ratio among the top 15 countries, signalling the burden it faced despite not leading in absolute case counts.

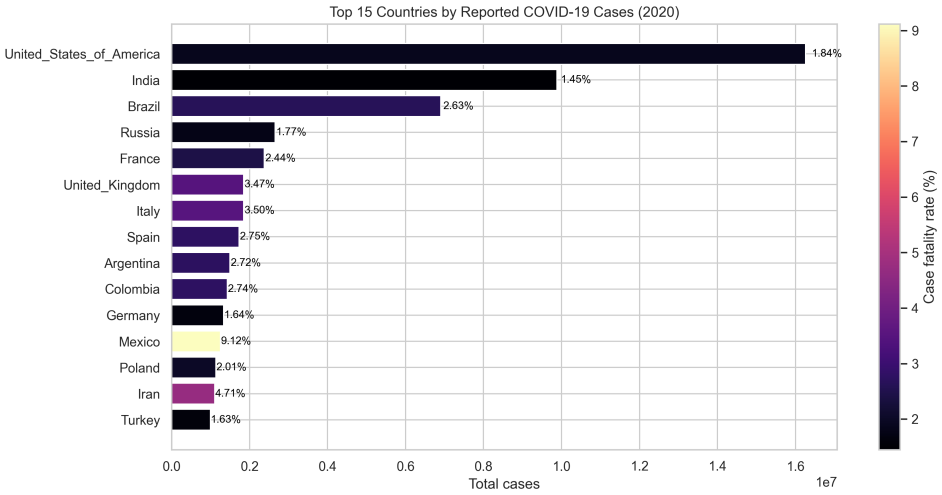


Figure 6: Top 15 countries. CFR annotations show that Mexico has the highest fatality ratio among the top group.

Figure 7 shifts the perspective to continents using monthly aggregates. The visualization script groups records by continent and month, orders them chronologically, and builds a stacked area chart so that each band represents one continent's contribution over time. The resulting waveform shows Asia contributing the first wave in early 2020, Europe cresting twice (spring and autumn), and the Americas dominating the mid-year surge before the global plateau in December.

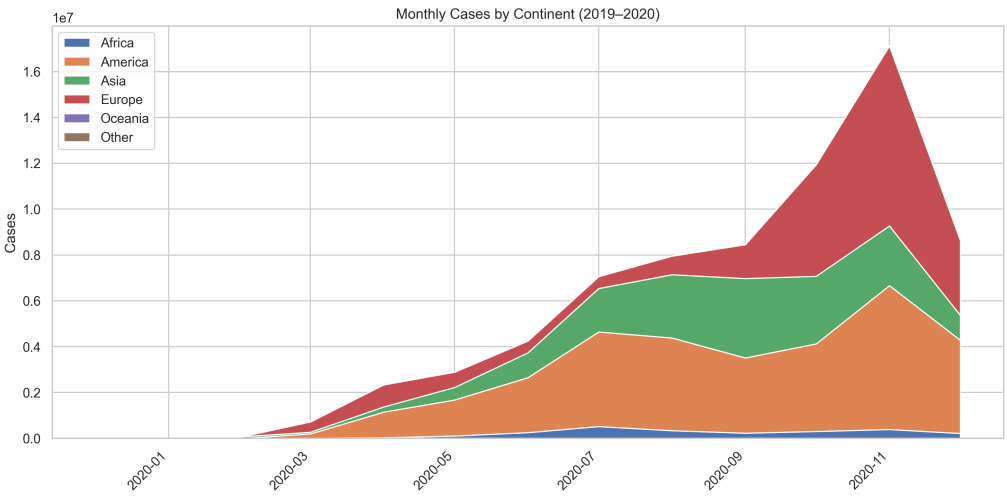


Figure 7: Monthly cases by continent. Asia initiates the pandemic, Europe delivers two waves, and the Americas dominate mid-2020.

Figure 8 aggregates the dataset by calendar month (using the parsed `year` and `month` fields) and renders the results with dual axes: blue columns for cases on the left axis, and a red line for deaths on the right axis. This side-by-side scaling makes the relative timing clearer while preserving the original magnitudes—the death curve still trails cases by about four weeks.

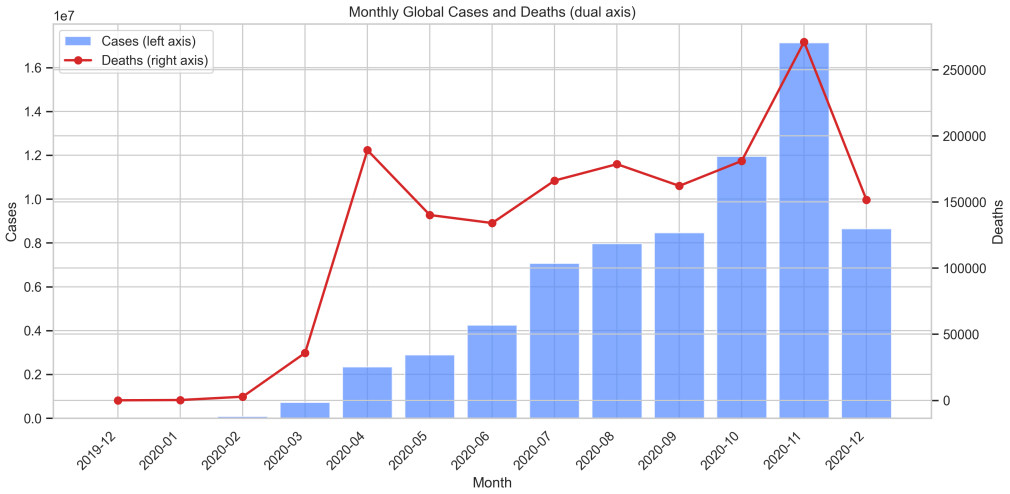


Figure 8: Monthly global cases (bars, left axis) and deaths (line, right axis). The mortality wave follows the surge in cases by roughly one month.

Figure 9 operates at the country level again but focuses on intensity rather than cumulative totals. The MapReduce and Spark jobs compute rolling 14-day sums per country; the visualization script keeps only the maximum value for each territory and pivots the top 30 results into a heatmap. The palette highlights the extremes: the United States crosses 2.8 million cases over its worst 14-day window, India peaks at roughly 1.3 million, and France, Brazil, and Italy follow in the next tier. Countries lower down the list remain almost black, signaling that their surges never approached the same magnitude.

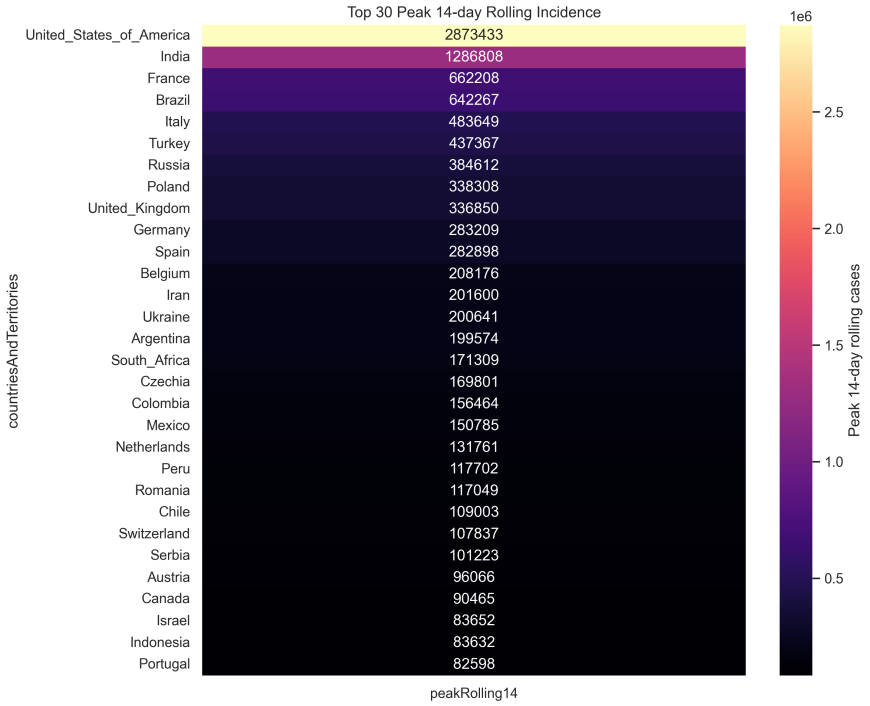
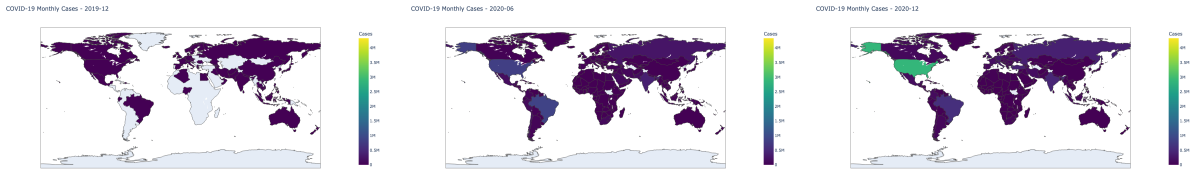


Figure 9: Peak 14-day rolling incidence for the top 30 territories. The colour scale emphasises the three highest peaks (United States, India, France) while the remaining countries fade toward black as their rolling totals decline.

Figure 10 compares three snapshots of the pandemic: the quiet baseline at the end of 2019, the explosive mid-2020 surge centred in the Americas, and the widespread December 2020 phase where both hemispheres remain active. Reading the heat levels from left to right shows how quickly the epicentre shifted across continents while the gradient between low-incidence regions and global hotspots persisted.



(a) (b) (c)

Figure 10: (a) December 2019: cases concentrated in East Asia. (b) Mid-2020: Americas become the dominant hotspot. (c) December 2020: widespread activity across both hemispheres.

The full monthly animation is included in the submission (`Project3/visualizations/covid_animation.mp4`) and mirrored online at this repository link.

6 Key indicators and performance

6.1 Top countries (Dec 2019 to Dec 2020)

Table 2: Top countries by cumulative cases (subset)

Country	Cases	CFR (%)	Cases / 100k
United_States_of_America	16 256 754	1.84	4 940.29
India	9 884 100	1.45	723.36
Brazil	6 901 952	2.63	3 270.30
Russia	2 653 928	1.77	1 819.35
France	2 376 852	2.44	3 546.86
United_Kingdom	1 849 403	3.47	2 774.92

6.2 Performance comparison

Table 3: Average runtimes (three executions on macOS M1 Pro)

Engine	Job	Runtime (s)
MongoDB MapReduce	Country totals	0.37
MongoDB MapReduce	Peak 14-day	0.47
Hadoop Streaming	Country totals	20.07
Hadoop Streaming	Peak 14-day	19.99
Spark (local[4])	Aggregations	7.97

MongoDB proved ideal for fast exploration and quick tweaks inside the playground (both MapReduce jobs complete in well under half a second). Hadoop demanded more scaffolding but delivered fine-grained control that mirrors the exercises from class; the streaming jobs now average roughly twenty seconds each because the Python mapper/reducer start-ups dominate the runtime. Spark sits between the two extremes: it finishes the three aggregations in about eight seconds, writes the results as Parquet, and echoes the total duration using the built-in timer added to the script—a ranking consistent with published Spark-versus-MapReduce comparisons on analytical workloads [2, 5].

7 Discussion and conclusion

Running the three pipelines side by side did more than just stress the software. It confirmed that the underlying indicators are stable: the United States, India, and Brazil lead the global case count, Europe is on track for a significant resurgence in the autumn of 2020, mortality curves trail cases by about one month, and only a few territories have reached the most severe 14-day incidence levels. That consistency gave me the assurance that each platform was properly set up before moving on to the next.

Each platform has its own merits. MongoDB provided the quickest iteration cycle, therefore it was used to validate calculations and explore the dataset. Hadoop Streaming felt challenging to launch, but it followed the precise MapReduce patterns discussed in the tutorials, which confirmed that I could still describe the logic with explicit key-value programs in Python—just at the cost of longer wall-clock times. Spark remained a pragmatic compromise: the source stayed concise, the DataFrame API made the transformations understandable, the new runtime print-out reports an eight-second turnaround, and the outputs were immediately reusable thanks to Parquet storage.

If I had more time, I would look into filling the early gaps in the case and death columns, layering in hospitalisation or vaccination context, and turning the notebook-style scripts into scheduled runs. For now, the workflow already meets the course objectives and remains faithful to the toolset we practised in class while producing a complete, reproducible study of the ECDC dataset.

References

[1] ECDC. *Download Historical Data (to 14 December 2020) on the Daily Number of New Reported COVID-19 Cases and Deaths Worldwide*. https://www.ecdc.europa.eu/en/publications-data/download-todays-data-geographic-distribution-covid-19-cases-worldwide. Dec. 2020. (Visited on 11/01/2025).

[2] Taha Tekdogan and Ali Cakmak. "Benchmarking Apache Spark and Hadoop MapReduce on Big Data Classification". In: *2021 5th International Conference on Cloud and Big Data Computing (ICCBDC)*. Aug. 2021, pp. 15–20. DOI: 10.1145/3481646.3481649. arXiv: 2209.10637 [cs]. (Visited on 11/01/2025).

[3] MongoDB. *Map-Reduce - Database Manual - MongoDB Docs*. https://www.mongodb.com/docs/manu
reduce/. 2024. (Visited on 11/01/2025).

[4] The Apache Software Foundation. *Apache Hadoop 3.4.2 – MapReduce Tutorial*. https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html. Aug. 2025. (Visited on 11/01/2025).

[5] Ankush Verma, Ashik Hussain Mansuri, and Neelesh Jain. "Big Data Management Processing with Hadoop MapReduce and Spark Technology: A Comparison". In: *2016 Symposium on Colossal Data Analysis and Networking (CDAN)*. Mar. 2016, pp. 1–4. DOI: 10.1109/CDAN.2016.7570891. (Visited on 11/01/2025).

Appendix A: Reproduce the pipeline

A.1 MongoDB

1. Launch `mongod`.
2. Run `mongosh < MongoDB/health_data.mongodb.js`.
3. Export analytics with `mongoexport` if needed.

A.2 Hadoop Streaming

1. Enter the Docker container `rdurandealba/hadoop_mac_class`.
2. Stage NDJSON into HDFS and make scripts executable (Section 4.2).
3. Run the two streaming jobs and inspect the outputs.

A.3 Spark

1. Ensure `spark-submit` is in `PATH`.
2. Launch the job using:

```
spark-submit Spark/spark_health_analysis.py INPUT_FIELD GROUP_FIELD
    OUTPUT_DIR
```

Listing 6: Spark submission helper

A.4 Visualisations

1. Install `pandas`, `matplotlib`, `seaborn` (and optionally `plotly`, `ffmpeg`).
2. Regenerate the figures and animation:

```
python visualizations/build_visualizations.py --input Project3/data
   /encounters.ndjson
```

Listing 7: Visualisation rebuild command

3. Outputs are written to `Project3/visualizations/outputs` (video: `Project3/visualizations/covid_animation.mp4`).